

Stratégie de Tests - JobbingTrack

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)

Documentation de la stratégie et méthodologie de tests pour JobbingTrack v4.1.

Vue d'ensemble

JobbingTrack utilise une approche complète de tests multi-niveaux couvrant tests unitaires, d'intégration, end-to-end, performance et sécurité.

Types de Tests

1. Tests Unitaires

Tests isolés de fonctions et composants individuels.

Objectif: Vérifier le bon fonctionnement de chaque unité de code en isolation.

Frameworks:

- **Backend:** Jest
- **Frontend:** Jest + React Testing Library
- **Coverage:** > 80%

Exemples:

```
// Test d'une fonction utilitaire
describe('formatDate', () => {
  it('devrait formater correctement une date', () => {
    const date = new Date('2025-01-15');
    expect(formatDate(date)).toBe('15/01/2025');
  });
});

// Test d'un composant React
describe('Button', () => {
  it('devrait appeler onClick au clic', () => {
    const onClick = jest.fn();
    render(<Button onClick={onClick}>Click me</Button>);
    fireEvent.click(screen.getByText('Click me'));
    expect(onClick).toHaveBeenCalledTimes(1);
  });
});
```

2. Tests d'Intégration

Tests des interactions entre plusieurs composants/services.

Objectif: Vérifier que les différents modules fonctionnent ensemble correctement.

Portée:

- Communication inter-services
- Intégration base de données
- API endpoints
- Flux de données

Exemples:

```
// Test API + Database
describe('POST /applications', () => {
  it('devrait créer une candidature et la persister en BDD', async () => {
    const response = await request(app)
      .post('/applications')
      .set('Authorization', `Bearer ${token}`)
      .send({ companyId: '123', position: 'Developer' });

    expect(response.status).toBe(201);

    const application = await db.applications.findById(response.body.id);
    expect(application).toBeDefined();
    expect(application.position).toBe('Developer');
  });
});
```

3. Tests End-to-End (E2E)

Tests de scénarios utilisateur complets via l'interface.

Objectif: Valider les parcours utilisateur du début à la fin.

Framework: Playwright

Navigateurs testés:

- Chrome/Chromium
- Firefox
- Safari
- Mobile (Chrome/Safari)

Scénarios:

```
// Test parcours complet candidature
test('création complète d\'une candidature', async ({ page }) => {
  // 1. Connexion
  await page.goto('http://localhost:8080/login');
  await page.fill('[name="email"]', 'user@example.com');
  await page.fill('[name="password"]', 'password123');
  await page.click('button[type="submit"]');

  // 2. Navigation vers candidatures
  await page.click('text=Candidatures');
  await page.click('text=Nouvelle candidature');

  // 3. Remplissage formulaire
  await page.fill('[name="position"]', 'Développeur Full Stack');
  await page.selectOption('[name="companyId"]', '123');
  await page.fill('[name="notes"]', 'Candidature spontanée');

  // 4. Soumission
  await page.click('button:has-text("Créer")');

  // 5. Vérification
  await expect(page.locator('text=Candidature créée avec succès')).toBeVisible
});
```

4. Tests de Performance

Tests de charge et de stress du système.

Objectif: Mesurer les performances et identifier les goulots d'étranglement.

Métriques:

- Temps de réponse API (< 200ms p95)
- Throughput (requêtes/seconde)
- Utilisation ressources (CPU/RAM)
- Temps de rendu frontend

Outils:

- k6 (load testing)
- Lighthouse (performances web)
- Artillery (API load testing)

Exemple:

```
// k6 load test
import http from 'k6/http';
import { check, sleep } from 'k6';
```

```

export let options = {
  stages: [
    { duration: '2m', target: 100 }, // Montée à 100 users
    { duration: '5m', target: 100 }, // Maintien 100 users
    { duration: '2m', target: 0 },   // Descente à 0
  ],
};

export default function () {
  let res = http.get('http://localhost:3000/api/applications');
  check(res, {
    'status is 200': (r) => r.status === 200,
    'response time < 200ms': (r) => r.timings.duration < 200,
  });
  sleep(1);
}

```

5. Tests de Sécurité

Tests des vulnérabilités et vérifications sécurité.

Objectif: Identifier et corriger les failles de sécurité.

Vérifications:

- Injection SQL
- XSS (Cross-Site Scripting)
- CSRF (Cross-Site Request Forgery)
- Authentification/Autorisation
- Variables d'environnement sécurisées
- Dépendances vulnérables

Outils:

- OWASP ZAP
- npm audit
- Snyk
- Tests manuels

Exemple:

```

// Test injection SQL
describe('Security - SQL Injection', () => {
  it('devrait bloquer les tentatives d\'injection SQL', async () => {
    const maliciousInput = ''; DROP TABLE users; --";
    const response = await request(app)
      .get(`/api/users?search=${maliciousInput}`)
      .set('Authorization', `Bearer ${token}`);

    expect(response.status).not.toBe(500);
  });
});

```

```
// Vérifier que la table existe toujours
const users = await db.users.findAll();
expect(users).toBeDefined();
});
});
```

Outils et Frameworks

Backend

Outil	Usage	Version
Jest	Tests unitaires/intégration	^29.0.0
Supertest	Tests API HTTP	^6.3.0
@faker-js/faker	Données de test	^8.0.0

Frontend

Outil	Usage	Version
Jest	Tests unitaires	^29.0.0
React Testing Library	Tests composants	^14.0.0
@playwright/test	Tests E2E	^1.40.0

Performance

Outil	Usage	Version
k6	Load testing	latest
Lighthouse	Performances web	latest
Artillery	API load testing	^2.0.0

Sécurité

Outil	Usage	Version
OWASP ZAP	Scan vulnérabilités	latest

npm audit	Audit dépendances	built-in
Snyk	Scan sécurité	latest



Coverage et Rapports

Objectifs de Coverage

- **Global:** > 80%
- **Backend services critiques:** > 90%
- **Frontend composants:** > 75%
- **Intégration:** > 70%

Rapports Générés

```
tests/reports/
├─ test-report.json      # Rapport principal
├─ junit-results.xml     # CI/CD
├─ performance-report.json # Performance
├─ security-report.json  # Sécurité
└─ playwright-report/   # E2E HTML

tests/coverage/
├─ index.html           # Coverage HTML
├─ lcov.info            # LCOV format
└─ coverage-summary.json # JSON summary
```



Commandes

Commandes principales

```
# Configuration initiale
make test-setup

# Tous les tests
make test-all

# Tests rapides (sans E2E)
make test-quick
```

```
# Tests avec coverage  
make test-coverage
```

Tests par catégorie

```
make test-unit           # Tests unitaires  
make test-integration    # Tests intégration  
make test-database       # Tests BDD  
make test-api            # Tests API  
make test-backend        # Tests backend  
make test-frontend       # Tests frontend  
make test-e2e            # Tests E2E  
make test-performance    # Tests performance  
make test-security       # Tests sécurité
```

Nettoyage

```
make test-clean          # Nettoyer environnement tests  
make test-clean-reports # Nettoyer rapports
```

Intégration Continue (CI/CD)

GitHub Actions Workflow

```
name: Tests  
  
on: [push, pull_request]  
  
jobs:  
  test:  
    runs-on: ubuntu-latest  
  
    steps:  
      - uses: actions/checkout@v3  
  
      - name: Setup Node.js  
        uses: actions/setup-node@v3  
        with:  
          node-version: '20'  
  
      - name: Install dependencies
```

```

    run: npm install

- name: Run tests
  run: make test-all

- name: Upload coverage
  uses: codecov/codecov-action@v3
  with:
    file: ./tests/coverage/lcov.info

```

Pipeline

```

1. Commit → 2. Tests unitaires → 3. Tests intégration
                        ↓                      ↓
4. Tests E2E ← 5. Build ← 6. Deploy staging
                        ↓
7. Tests acceptance → 8. Deploy production

```



Dépannage

Tests échouent

```

# Vérifier services
make health

# Consulter logs
make logs

# Redémarrer environnement
make down && make up

# Nettoyer et réinstaller
make test-clean && make test-setup

```

Tests E2E timeout

```

# Augmenter timeout dans playwright.config.ts
timeout: 60000 # 60 secondes

```



```
# Attendre services  
await page.waitForLoadState('networkidle');
```

Coverage insuffisant

```
# Voir fichiers non couverts  
make test-coverage  
open tests/coverage/index.html  
  
# Ajouter tests pour fichiers critiques
```



Ressources

- [Tests Racine](#) - Documentation complète des tests
- [Guide Développement](#) - Configuration environnement tests
- [API Reference](#) - Documentation API pour tests
- [Guide CI/CD](#) - Pipeline déploiement



Checklist Avant Commit

- ☐ Tests unitaires passent (`make test-unit`)
- ☐ Tests intégration passent (`make test-integration`)
- ☐ Coverage > 80% maintenu
- ☐ Pas de console.log/console.error oubliés
- ☐ Tests ajoutés pour nouveaux features
- ☐ Tests mis à jour pour modifications
- ☐ Linter sans erreurs
- ☐ Build réussit



Checklist Avant Déploiement

- ☐ Tous tests passent (`make test-all`)
- ☐ Tests E2E passent (`make test-e2e`)
- ☐ Tests performance acceptables
- ☐ Tests sécurité OK
- ☐ Pas de dépendances vulnérables (`npm audit`)
- ☐ Documentation à jour
- ☐ Changelog mis à jour

Version: 4.1

Dernière mise à jour: Octobre 2025